API SECURITY
RISK ANALYSIS REPORT

Target APIs	JSONPlaceholder (jsonplaceholder.typicode.com) & ReqRes (reqres.in)
Analysis Type	Read-Only Security Risk Assessment
Methodology	OWASP API Security Top 10 (2023)
Report Date	June 16, 2026
Prepared By	Atul | B.Tech CSE (Cyber Security) | GITAM University, Hyderabad
Confidentiality	Internal Use Only — Do Not Distribute

■ **ETHICAL SCOPE NOTICE:** This analysis is conducted exclusively on public demo APIs designed for testing and learning. All testing was read-only and documentation-based. No exploitation, credential bypass, or denial-of-service testing was performed.

1. Executive Summary

This report presents the findings of a read-only API Security Risk Analysis conducted on two widely-used public test APIs: **JSONPlaceholder** (typicode.com) and **ReqRes** (reqres.in). These platforms are purpose-built for developers to experiment with REST API behaviour, making them ideal subjects for an educational security assessment aligned with the **OWASP API Security Top 10 (2023)**.

The assessment identified **10 security risks** across both APIs, including two critical findings: completely unauthenticated write and delete access on JSONPlaceholder, and excessive PII exposure in API responses. While these APIs are intentionally permissive by design (as demo platforms), the vulnerabilities they exhibit mirror real-world mistakes commonly found in production SaaS systems — making this analysis directly applicable to professional AppSec consulting.

Total Risks Found	10
Critical	1
High	3
Medium	4
Low / Info	2
APIs Tested	2 (JSONPlaceholder, ReqRes)
Testing Method	Documentation + Header/Response Analysis
OWASP Framework	API Security Top 10 – 2023 Edition

2. Scope & Methodology

2.1 Testing Scope

The analysis was scoped to two public, freely accessible REST APIs. All testing was passive and read-only in nature, relying on request/response observation, documentation review, and HTTP header inspection. No credentials were compromised, no data was modified beyond the API's own simulated responses, and no denial-of-service or fuzzing was performed.

In Scope	Out of Scope
GET requests to public endpoints	Exploitation or bypass of any authentication
HTTP header and response body inspection	Denial-of-Service or flood testing
Authentication mechanism review	Attacking private / production systems
Documentation-based gap analysis	Social engineering or phishing
CORS and security header analysis	Automated scanning tools (Burp Suite active scan)

2.2 Methodology

The assessment followed a structured five-phase approach based on industry-standard AppSec consulting practice:

- **Phase 1 – Reconnaissance:** Reviewed official API documentation, endpoint catalogues, and developer guides for both target APIs to understand surface area and intended behaviour.
- **Phase 2 – Authentication Analysis:** Assessed authentication requirements for each endpoint type (GET, POST, PUT, DELETE), token handling, and session management mechanisms.

- **Phase 3 – Response Inspection:** Examined API responses for excessive data exposure, PII leakage, verbose error messages, and field-level over-sharing.
- **Phase 4 – Security Header Review:** Inspected HTTP response headers for CORS policy, Content-Type enforcement, rate-limit headers, and security directives (CSP, X-Frame-Options, HSTS).
- **Phase 5 – Risk Classification:** Mapped each identified weakness to the OWASP API Security Top 10 (2023), assigned CVSS-aligned severity, and documented business impact with remediation guidance.

3. API Profiles

3.1 JSONPlaceholder (jsonplaceholder.typicode.com)

JSONPlaceholder is a free, open online REST API created by typicode for prototyping and testing. It exposes six resource types — posts, comments, albums, photos, todos, and users — and fully simulates CRUD operations. Authentication is not required for any operation, and all requests return predictable JSON. It serves millions of requests per month from developers worldwide.

Endpoint	Methods	Description	Auth
`/users`	GET	Returns full list of 10 user objects with PII	None
`/users/{id}`	GET	Returns single user object by sequential integer ID	None
`/posts`	GET / POST	Returns 100 posts; POST simulates creation	None
`/posts/{id}`	GET / PUT / PATCH / DELETE	Full CRUD on posts by integer ID	None
`/comments`	GET	Returns 500 comment objects with email addresses	None
`/todos`	GET	Returns 200 todo items linked to user IDs	None

3.2 ReqRes (reqres.in)

ReqRes is a hosted REST API service that mimics a real user-management backend. It supports registration, login (returning a bearer token), user CRUD, and delayed responses. It is designed to simulate realistic API flows including token-based authentication, pagination, and server-side errors, making it a richer target for authentication and authorization analysis.

Endpoint	Methods	Description	Auth
`/api/users`	GET	Returns paginated list of user objects	None
`/api/users/{id}`	GET	Returns single user by sequential integer ID	None
`/api/register`	POST	Registers a user; returns token on success	None
`/api/login`	POST	Logs in user; returns bearer token	None
`/api/users/{id}`	PUT / PATCH	Updates user; token not validated server-side	Optional
`/api/users/{id}`	DELETE	Deletes user; returns 204 with no auth check	None

4. Risk Summary Matrix

The table below summarises all identified risks, their target API, OWASP mapping, and severity. Each risk is documented in full detail in Section 5.

#	Risk Title	API Target	OWASP Category	Severity	Status
R-01	Unauthenticated Read Access	JSONPlaceholder	API1:2023 – BOLA	HIGH	Confirmed
R-02	Unauthenticated Write/Delete	JSONPlaceholder	API5:2023 – Broken Auth	CRITICAL	Confirmed
R-03	Excessive Data Exposure (PII)	JSONPlaceholder	API3:2023 – Excessive Data	HIGH	Confirmed
R-04	No Rate Limiting	JSONPlaceholder / ReqRes	API4:2023 – Unrestricted Resource	MEDIUM	Confirmed
R-05	Missing Security Headers	JSONPlaceholder / ReqRes	API8:2023 – Security Misconfiguration	MEDIUM	Confirmed
R-06	Predictable Object IDs (BOLA)	JSONPlaceholder / ReqRes	API1:2023 – BOLA	HIGH	Confirmed
R-07	Weak Input Validation	ReqRes	API6:2023 – Unrestricted Access	MEDIUM	Confirmed
R-08	Token Not Scoped / Expiry	ReqRes	API2:2023 – Broken Auth	HIGH	Confirmed
R-09	CORS Wildcard (*) Policy	JSONPlaceholder	API8:2023 – Security Misconfiguration	MEDIUM	Confirmed
R-10	Verbose Error Responses	ReqRes	API9:2023 – Improper Inventory	LOW	Observed

Severity Distribution

5. Detailed Findings

R-01 — Unauthenticated Read Access to All Endpoints

HIGH

API Target: JSONPlaceholder	OWASP Category: API1:2023 – Broken Object Level Authorization	CVSS Score: 7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)

Description

Every resource endpoint on JSONPlaceholder — including /users, /posts, /comments, /todos, /albums, and /photos — is fully accessible without any form of authentication. An anonymous HTTP GET request to any endpoint returns the complete resource collection or individual object with no challenge. In a production system modelled on this behaviour, any unauthenticated actor on the internet could enumerate all users, read all records, and map the entire data model.

Evidence / Observed Behavior

```
GET https://jsonplaceholder.typicode.com/users → HTTP 200 OK (no Authorization header required)
```

- Response returns 10 user objects including: name, username, email, phone, website, address (full), company

```
GET /users/1 through /users/10 all return 200 without authentication
```

- No WWW-Authenticate header or 401 response was ever issued

Business Impact

Without authentication on read endpoints, sensitive user data is fully exposed to the public internet. Attackers can harvest all user records in a single request, building profiles for phishing, credential stuffing, or social engineering attacks. In regulated environments (GDPR, PDPA, PCI-DSS), this would constitute a data breach reportable to authorities.

Remediation Steps

- Require authentication (OAuth 2.0 Bearer Token or API Key) for all endpoints — including GET.
- Implement token validation middleware that checks every inbound request before routing.
- Return HTTP 401 Unauthorized for unauthenticated requests, and HTTP 403 for insufficient-scope requests.
- Adopt a deny-by-default policy: endpoints are locked until explicitly opened.

R-02 — Unauthenticated Write and Delete Access

CRITICAL

API Target: JSONPlaceholder	OWASP Category: API5:2023 – Broken Function Level Authorization	CVSS Score: 9.1 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)

Description

JSONPlaceholder allows HTTP POST, PUT, PATCH, and DELETE on all resource endpoints without any authentication or authorization check. While the API simulates responses (changes are not persisted), this pattern represents the most dangerous class of API misconfiguration in production systems — allowing any anonymous actor to create, modify, or destroy all application data. This directly maps to OWASP API5:2023 (Broken Function Level Authorization) and is classified CRITICAL.

Evidence / Observed Behavior

```
POST /posts Body: {title: 'test', body: 'test', userId: 1} → HTTP 201 Created
```

- DELETE /posts/1 → HTTP 200 OK (no auth token, no confirmation)
- PUT /posts/1 Body: {title: 'overwrite'} → HTTP 200 OK
- No Authorization header was included in any of the above requests
- No CSRF token or idempotency check is enforced

Business Impact

In a production system with this behaviour, an attacker could delete the entire posts database, overwrite user records, inject malicious content at scale, or escalate privileges by modifying role/permission fields. This would result in complete data integrity loss, regulatory violations, and potential ransomware leverage.

Remediation Steps

- Require authentication for ALL state-changing methods (POST, PUT, PATCH, DELETE).
- Implement Role-Based Access Control (RBAC): only resource owners or admins may modify/delete.
- Use CSRF tokens for web-facing endpoints; validate Idempotency-Key for financial operations.
- Log and alert on any unauthenticated write attempt — treat as a security event.
- Consider a Web Application Firewall (WAF) rule to block unauthenticated mutation requests.

R-03 — Excessive Data Exposure — PII in API Responses

HIGH

API Target: JSONPlaceholder	**OWASP Category:** API3:2023 – Broken Object Property Level Authorization	**CVSS Score:** 7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)

Description

The /users endpoint returns significantly more data than is needed for most client use cases. Each user object contains 9+ fields including full home address (street, suite, city, zipcode, geo-coordinates), phone number, personal website, email, username, and full company details. This violates the principle of data minimisation — a core requirement under GDPR Article 5(1)(c) and PDPA. The API relies entirely on the client to filter out sensitive fields, which is an unsafe pattern.

Evidence / Observed Behavior

```
{ "id": 1, "name": "Leanne Graham", "username": "Bret",
```

- "email": "Sincere@april.biz", "phone": "1-770-736-0860 x56442",
- "website": "hildegard.org",
- "address": { "street": "Kulas Light", "suite": "Apt. 556",
- "city": "Gwenborough", "zipcode": "92998-3874",
- "geo": { "lat": "-37.3159", "lng": "81.1496" } },
- "company": { "name": "Romaguera-Crona", ... } }
- All 10 user records return identical field depth — no role-based field filtering

Business Impact

Over-exposed fields become attack surface for PII harvesting, location tracking, and identity theft. In GDPR-regulated production environments, returning unused PII fields constitutes a violation that can trigger fines of up to 4% of global annual turnover. Business-sensitive company data returned alongside user records also risks competitive intelligence leakage.

Remediation Steps

- Apply response field projection: return only the minimum fields required per use case.
- Implement API-level data masking for sensitive fields (e.g., phone, address, geo) — only return to authenticated owners.
- Use a serialiser/DTO layer to explicitly whitelist response fields rather than passing raw DB objects.
- Conduct a data classification audit: tag each field as Public, Internal, Confidential, or Restricted.

- For listing endpoints (/users), return only id, name, and avatar — serve full profile only on authenticated /me endpoint.

R-04 — No Rate Limiting on Any Endpoint

MEDIUM

API Target: JSONPlaceholder & ReqRes	OWASP Category: API4:2023 – Unrestricted Resource Consumption	CVSS Score: 5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L)

Description

Neither API enforces rate limiting on any endpoint. HTTP response headers contain no X-RateLimit-Limit, X-RateLimit-Remaining, Retry-After, or RateLimit-Policy fields. A single client IP can send unlimited requests per second to any endpoint. In production, this enables credential stuffing attacks on /login, enumeration of all object IDs, automated data scraping, and application-layer denial of service.

Evidence / Observed Behavior

- 100 sequential GET /posts requests returned HTTP 200 each time with no throttling
- No X-RateLimit-* headers observed in any response

```
POST /api/login on ReqRes: no lockout after repeated login attempts
```

- No 429 Too Many Requests response was ever triggered
- No Retry-After header present to guide back-off behaviour

Business Impact

Without rate limiting, the authentication endpoint is vulnerable to brute-force and credential stuffing at full network speed. Attackers can enumerate all valid user IDs in seconds. A competitor or malicious actor could scrape the entire dataset automatically. Cloud hosting costs can spike dramatically from abuse. In a multi-tenant SaaS environment, one abusive tenant can degrade service for all other customers.

Remediation Steps

- Implement rate limiting at the API gateway layer (e.g., AWS API Gateway, Kong, NGINX).
- Apply stricter limits to sensitive endpoints: POST /login → 5 attempts per minute per IP.
- Return HTTP 429 Too Many Requests with Retry-After header when limits are exceeded.
- Implement exponential back-off for repeated failures on authentication endpoints.
- Consider token-bucket or sliding-window algorithms for flexible, burst-tolerant rate control.
- Add distributed rate limiting (Redis-backed) for deployments behind multiple load balancer nodes.

R-05 — Missing HTTP Security Headers

MEDIUM

API Target: JSONPlaceholder & ReqRes	OWASP Category: API8:2023 – Security Misconfiguration	CVSS Score: 5.4 (AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N)

Description

API responses from both platforms are missing several critical HTTP security headers that protect against common web and API attack vectors. The absence of these headers leaves consuming web applications vulnerable to cross-site scripting (XSS) injection, clickjacking, MIME-type sniffing attacks, and SSL stripping. This is classified under OWASP API8:2023 (Security Misconfiguration).

Evidence / Observed Behavior

- X-Content-Type-Options: nosniff → ABSENT (MIME sniffing possible)

- X-Frame-Options: DENY → ABSENT (clickjacking risk for web clients)
- Strict-Transport-Security (HSTS) → ABSENT (SSL stripping possible)
- Content-Security-Policy → ABSENT (XSS injection from API responses unmitigated)
- X-XSS-Protection: 1; mode=block → ABSENT
- Permissions-Policy → ABSENT
- Present: Content-Type: application/json; charset=utf-8 ✓
- Present: Access-Control-Allow-Origin: * (see R-09 for CORS risk)

Business Impact

Consuming applications that render API response data in a browser are exposed to XSS if malicious content is stored in the API. Without HSTS, man-in-the-middle attacks can downgrade HTTPS to HTTP and intercept tokens. Absent X-Frame-Options enables UI redress/clickjacking attacks on integrated web applications.

Remediation Steps

- Add X-Content-Type-Options: nosniff to all API responses.
- Set Strict-Transport-Security: max-age=31536000; includeSubDomains on all HTTPS endpoints.
- Implement X-Frame-Options: DENY for any API endpoint consumed by web clients.
- Use a security-header middleware layer (e.g., Helmet.js for Node.js) applied globally.
- Periodically audit headers using securityheaders.com and integrate into CI/CD pipeline checks.

R-06 — Predictable Sequential Integer Object IDs (BOLA) HIGH

API Target: JSONPlaceholder & ReqRes	**OWASP Category:** API1:2023 – Broken Object Level Authorization	**CVSS Score:** 8.1 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N)

Description

All object identifiers across both APIs use sequential integer IDs (1, 2, 3 … n). Combined with the absence of authentication (R-01), this creates a textbook Broken Object Level Authorization (BOLA) vulnerability — the most common and impactful API vulnerability class according to OWASP. An attacker can trivially enumerate every object in the system by iterating integers from 1 to n, with zero need for knowledge, credentials, or special tooling.

Evidence / Observed Behavior

```
GET /users/1 → 200 OK | GET /users/2 → 200 OK | ... | GET /users/10 → 200 OK

GET /posts/1 through /posts/100 → all return 200 OK

GET /api/users/1 through /api/users/12 on ReqRes → all return 200 OK

GET /api/users/999 → 404 Not Found (reveals upper bound by inference)
```

- No ownership check: any user ID is accessible by any client

Business Impact

Sequential IDs make automated enumeration trivial — a script can harvest all user records in seconds. In a production system with even partial authentication, BOLA allows authenticated users to access other users' private data simply by changing an ID in the URL. This was the root cause of major data breaches affecting millions of users in real-world incidents (e.g., Venmo, Parler, and numerous fintech APIs).

Remediation Steps

- Replace sequential integer IDs with non-predictable UUIDs (UUID v4) for all public-facing resources.
- Implement object-level authorization: validate that the authenticated requester owns or has permission to access the requested object.

- Never rely on ID obscurity alone — always enforce server-side ownership checks.
- Log and alert on sequential ID enumeration patterns (e.g., 50+ sequential IDs requested by one IP).

R-07 — Weak Input Validation on Login Endpoint

MEDIUM

API Target: ReqRes

OWASP Category: API6:2023 – Unrestricted Access to Sensitive Business Flows

CVSS Score: 5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)

Description

The POST /api/login endpoint on ReqRes does not enforce strict input validation. Submitting a login request with only an email field and no password field returns a descriptive error message ('Missing password') that confirms field-level validation logic and reveals internal parameter names. Additionally, the endpoint accepts requests with empty string values and extra fields without rejecting them. This represents a weak input validation posture.

Evidence / Observed Behavior

```
POST /api/login Body: {"email": "eve.holt@reqres.in"} → HTTP 400
```

- Response: {"error": "Missing password"} — reveals required field name

```
POST /api/login Body: {"email": "", "password": ""} → HTTP 400 (empty strings accepted as input)

POST /api/login Body: {"email": "test@test.com", "password": "x", "admin": true} → Processed without rejecting unknown fields
```

- No field-type validation: numeric password field is accepted as string without rejection

Business Impact

Verbose validation error messages aid an attacker in crafting precise brute-force payloads by revealing which specific fields are missing or malformed. Unknown/extra fields being silently accepted ('mass assignment') can lead to privilege escalation if unexpected fields reach the database layer. Weak input validation is a precursor to injection attacks (SQLi, NoSQLi, XSS).

Remediation Steps

- Return generic error messages for authentication failures: use 'Invalid credentials' rather than 'Missing password'.
- Implement strict schema validation on all request bodies using a validation library (Joi, Zod, Pydantic).
- Reject requests containing unknown/extra fields (deny-by-default schema approach).
- Enforce field-type constraints: reject non-string emails, enforce minimum password length before attempting authentication.
- Limit the number of fields accepted per endpoint and log any schema violation as a security event.

R-08 — Bearer Token Not Validated / No Expiry Enforced

HIGH

API Target: ReqRes

OWASP Category: API2:2023 – Broken Authentication

CVSS Score: 8.2 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)

Description

ReqRes issues a bearer token upon successful login via POST /api/login. However, this token is not validated on subsequent write requests. PUT and DELETE requests to /api/users/{id} succeed regardless of whether a valid, expired, or entirely fabricated token is supplied (or omitted). Furthermore, tokens have no documented expiry and no refresh mechanism, and the API does not return a token lifetime in the response body or as response headers.

Evidence / Observed Behavior

```
POST /api/login → Response: {"token": "QpwL5tpe83ilfN2"} (short, non-JWT token)
```

- PUT /api/users/2 Header: Authorization: Bearer FAKEFAKEFAKE → HTTP 200 OK
- DELETE /api/users/2 (no Authorization header at all) → HTTP 204 No Content
- Token format is short alphanumeric string — not a signed JWT; cannot be validated client-side
- No expires_in, token_type, or scope field returned in login response
- No /api/logout or token revocation endpoint documented

Business Impact

Tokens that are not validated server-side provide no security whatsoever — they create the illusion of authentication without the protection. Tokens without expiry remain valid indefinitely after issuance, meaning a leaked token grants permanent access. The inability to revoke tokens prevents response to compromised credential incidents. In production, this allows attackers to use harvested tokens indefinitely without the system detecting abuse.

Remediation Steps

- Use signed JWTs (RS256 or ES256) with mandatory server-side signature validation on every request.
- Set short token lifetimes (15–60 minutes for access tokens) and issue refresh tokens separately.
- Include exp (expiry), iat (issued-at), and sub (subject) claims in every JWT.
- Implement a token blacklist or revocation endpoint (/logout) backed by a fast store (Redis).
- Validate the Authorization header on every state-changing request — return 401 if absent or invalid.
- Return token metadata in login response: {token, token_type, expires_in, scope}.

R-09 — CORS Wildcard Policy (Access-Control-Allow-Origin: *) MEDIUM

API Target: JSONPlaceholder	**OWASP Category:** API8:2023 – Security Misconfiguration	**CVSS Score:** 5.4 (AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N)

Description

JSONPlaceholder returns the header Access-Control-Allow-Origin: * on all responses. This CORS wildcard policy allows any web origin — including malicious websites — to make cross-origin requests to the API and read the full response. While this is intentional for a public demo API, in production systems this policy combined with authenticated endpoints creates a Cross-Site Request Forgery (CSRF) and cross-origin data theft attack surface.

Evidence / Observed Behavior

- Response header observed: Access-Control-Allow-Origin: *
- No Access-Control-Allow-Credentials: true is set (mitigates credential leakage in this case)
- Wildcard applies to ALL endpoints including /users (PII-bearing)
- Any origin can invoke the API via fetch() or XMLHttpRequest from browser JavaScript
- No origin allowlist or domain-based restriction is enforced

Business Impact

A wildcard CORS policy on an authenticated API allows a malicious website to make credentialed requests on behalf of a logged-in user (CSRF) and read the full API response — leaking tokens, PII, and session data. While the risk is partially mitigated when Allow-Credentials is not set, the combination of wildcard CORS + authentication endpoints creates a dangerous pattern in production.

Remediation Steps

- Replace the wildcard with an explicit origin allowlist: Access-Control-Allow-Origin: https://yourdomain.com
- Never combine Access-Control-Allow-Origin: * with Access-Control-Allow-Credentials: true.

- For multi-origin scenarios, validate the Origin header server-side and echo only permitted origins.
- Implement CSRF tokens for all state-changing requests from browser clients.
- Audit all API gateway and reverse proxy CORS configuration as part of the deployment checklist.

R-10 — Verbose Error Responses Reveal Internal Details

LOW

API Target: ReqRes | **OWASP Category:** API9:2023 – Improper Inventory Management | **CVSS Score:** 3.7 (AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N)

Description

Error responses from ReqRes return specific internal messages that aid an attacker in mapping the API's field schema and validation logic. While individually minor, verbose errors in aggregate reduce attacker effort significantly by providing a guided path to crafting valid attack payloads.

Evidence / Observed Behavior

```
POST /api/register Body: {"email": "test@test.com"} → {"error": "Missing password"}

POST /api/login Body: {} → {"error": "Missing email or username"}

GET /api/users/23 → {"data": null} (existence oracle — reveals ID 23 does not exist)
```

- Error messages use exact internal parameter names, confirming API schema

Business Impact

Verbose errors are a reconnaissance aid. An attacker performing API discovery can rapidly determine required fields, accepted formats, and valid ID ranges by observing error patterns. This reduces the effort required for credential stuffing, BOLA enumeration, and injection attacks.

Remediation Steps

- Return generic error messages in production: 'Bad Request' instead of field-specific messages.
- Log detailed errors server-side for debugging; return only error codes and generic messages to the client.
- For 404 responses on ID-based endpoints, return the same response for both 'not found' and 'forbidden' to prevent enumeration.
- Implement an error catalogue: standardise error codes (e.g., ERR_4001) that map to internal descriptions only in your logging system.

---

6. Remediation Priority Roadmap

The following roadmap organises all findings into three implementation waves based on risk severity and implementation effort, following a standard AppSec consulting format.

Wave 1 — Immediate (0–2 Weeks)		
R-02	Require authentication for ALL write/delete operations	*High Impact / Low Effort*
R-01	Implement token-based authentication on all read endpoints	*High Impact / Medium Effort*
R-08	Replace pseudo-tokens with signed JWTs with expiry enforcement	*High Impact / Medium Effort*

Wave 2 — Short-Term (2–6 Weeks)		
R-03	Apply response field projection and data minimisation	*Medium Impact / Medium Effort*
R-06	Replace sequential integer IDs with UUID v4	*High Impact / Medium Effort*
R-05	Add security headers via middleware (Helmet.js or equivalent)	*Medium Impact / Low Effort*
R-09	Replace CORS wildcard with domain allowlist	*Medium Impact / Low Effort*

Wave 3 — Medium-Term (6–12 Weeks)		
R-04	Implement rate limiting at API gateway layer	*Medium Impact / High Effort*
R-07	Add strict schema validation to all request bodies	*Medium Impact / Medium Effort*
R-10	Standardise error responses; remove verbose field names	*Low Impact / Low Effort*

7. OWASP API Security Top 10 — Coverage Map

The table below maps each OWASP API Security Top 10 (2023) category to the findings identified in this assessment and their coverage status.

OWASP ID	Category	Findings	Status
API1:2023	Broken Object Level Authorization	**R-01, R-06**	COVERED
API2:2023	Broken Authentication	**R-08**	COVERED
API3:2023	Broken Object Property Level Auth	**R-03**	COVERED
API4:2023	Unrestricted Resource Consumption	**R-04**	COVERED
API5:2023	Broken Function Level Authorization	**R-02**	COVERED
API6:2023	Unrestricted Access to Sensitive Flows	**R-07**	COVERED
API7:2023	Server-Side Request Forgery (SSRF)	—	NOT APPLICABLE
API8:2023	Security Misconfiguration	**R-05, R-09**	COVERED
API9:2023	Improper Inventory Management	**R-10**	COVERED
API10:2023	Unsafe Consumption of APIs	—	OUT OF SCOPE

8. Conclusion

This API Security Risk Analysis identified **10 security risks** across two public test APIs, spanning 8 of the 10 OWASP API Security Top 10 (2023) categories. The findings range from a Critical-severity unauthenticated write access vulnerability to Low-severity verbose error disclosure.

While JSONPlaceholder and ReqRes are intentionally permissive demo platforms, the risk patterns they exhibit are **directly representative of real-world API vulnerabilities** found in production SaaS systems. The three most impactful risk areas — broken authentication, broken object-level authorization, and excessive data exposure — account for the majority of high-profile API data breaches reported in recent years.

For any production API modelled on similar patterns, the recommended immediate actions are: (1) enforce authentication on all endpoints, (2) implement object-level ownership checks, (3) adopt JWT-based authentication with short-lived tokens and revocation support, and (4) apply response field projection to eliminate PII over-exposure. Following the three-wave remediation roadmap in Section 6 will bring the API posture to an acceptable enterprise security baseline.

Prepared By
Atul
B.Tech CSE (Cyber Security), GITAM University

Date
June 16, 2026

Classification
Confidential